

DAY – 9

QUESTION 1

1. The Basic Assertion Goal: Understand the simplicity of pytest syntax. Write a file named `test\_logic.py`. Create a test function `test\_math\_operations` that uses simple `assert` statements to verify: That \$15 \times 3\$ equals \$45\$. That the string `pytest` is present within the phrase `Learning pytest is fun`.
2. CODE:

```
def test_math_operations():  
    assert 15 * 3 == 45  
    assert "pytest" in "Learning pytest is fun"
```

3. OUTPUT

Testing started at 11:39 PM ...

Launching pytest with arguments

C:\WiproTraining\python\PythonCoding\daily_assignments\test_logicday9.py --no-header --no-summary -q in C:\WiproTraining\python\PythonCoding

===== test session starts

=====

collecting ... collected 1 item

daily_assignments/test_logicday9.py::test_math_operations PASSED [100%]

===== 1 passed in 0.02s

=====

Process finished with exit code 0

QUESTION 2

Using Fixtures for Setup Goal: Replace `setUp` and `tearDown` with the modern `@pytest.fixture` approach. Create a pytest fixture named `sample\_dict` that returns a dictionary: `{"name": "Alice", "role": "Dev"}`. Write a test function `test\_dict\_keys` that: Accepts the fixture as an argument. Asserts that the key `role` exists in the dictionary. Asserts that the value of `name` is `Alice`.

CODE

```
import pytest  
@pytest.fixture  
def sample_dict():  
    return {"name": "Alice", "role": "Dev"}  
def test_dict_keys(sample_dict):  
    assert "role" in sample_dict  
    assert sample_dict["name"] == "Alice"
```

OUTPUT

Testing started at 11:41 PM ...

Launching pytest with arguments

C:\WiproTraining\python\PythonCoding\daily_assignments\test_logicques2day9.py --no-header --no-summary -q in C:\WiproTraining\python\PythonCoding

===== test session starts =====

collecting ... collected 1 item

daily_assignments/test_logicques2day9.py::test_dict_keys PASSED [100%]

===== 1 passed in 0.02s =====

Process finished with exit code 0

QUESTION 3

Handling Exceptions Goal: Use the `pytest.raises` context manager. Write a function `get\_element(my\_list, index)` that returns an item from a list. Write a test function `test\_index\_error` that uses `with pytest.raises(IndexError):` to verify that attempting to access index `10` of the list `[1, 2, 3]` correctly triggers an error.

CODE

```
import pytest
def get_element(my_list, index):
    return my_list[index]
def test_index_error():
    with pytest.raises(IndexError):
        get_element([1, 2, 3], 10)
```

OUTPUT

Testing started at 11:43 PM ...

Launching pytest with arguments

C:\WiproTraining\python\PythonCoding\daily_assignments\test_logicques3day9.py --no-header --no-summary -q in C:\WiproTraining\python\PythonCoding

===== test session starts =====

collecting ... collected 1 item

daily_assignments/test_logicques3day9.py::test_index_error PASSED [100%]

===== 1 passed in 0.02s =====

Process finished with exit code 0

QUESTION 4

Parameterized Testing Goal: Run the same test logic with multiple sets of data. Use the `@pytest.mark.parametrize` decorator to create a single test function `test\_is\_even`. Pass three different inputs to the test: `2`, `10`, and `22`. The test should assert that each input `% 2 == 0`. Observe how pytest treats these as three separate test cases in the output

CODE

```
import pytest
@pytest.mark.parametrize("num", [2, 10, 22])
def test_is_even(num):
    assert num % 2 == 0
```

OUTPUT

Testing started at 11:45 PM ...

Launching pytest with arguments daily_assignments/test_logicques4day9.py::test_is_even --no-header --no-summary -q in C:\WiproTraining\python\PythonCoding

===== test session starts =====

collecting ... collected 3 items

daily_assignments/test_logicques4day9.py::test_is_even[2] PASSED [33%]
daily_assignments/test_logicques4day9.py::test_is_even[10] PASSED [66%]
daily_assignments/test_logicques4day9.py::test_is_even[22] PASSED [100%]

===== 3 passed in 0.10s =====

Process finished with exit code 0

QUESTION 5

Cleaning Up with Fixture Yields Goal: Manage resources (like files or database connections) properly. Create a fixture named `temp\_file`. In the Setup phase : Create a new text file named `test.txt` and write `"Hello World"` into it. Yield the filename to the test. In the Teardown phase (after the yield): Use the `os` module to delete the file. Write a test `test\_file\_content` that reads the file and verifies the text matches

CODE

```
import pytest
import os
@pytest.fixture
def temp_file():
    filename = "test.txt"
    with open(filename, "w") as f:
        f.write("Hello World")
    yield filename
    if os.path.exists(filename):
        os.remove(filename)
def test_file_content(temp_file):
    with open(temp_file, "r") as f:
        content = f.read()
    assert content == "Hello World"
```

OUTPUT

Testing started at 11:47 PM ...

Launching pytest with arguments

C:\WiproTraining\python\PythonCoding\daily_assignments\test_lodicques5day9.py --no-header --no-summary -q in C:\WiproTraining\python\PythonCoding

===== test session starts =====

collecting ... collected 1 item

daily_assignments/test_lodicques5day9.py::test_file_content PASSED [100%]

===== 1 passed in 0.02s =====

Process finished with exit code 0